

How to Seamlessly Integrate **Stripe** Payments in **React Native?** 🚀

stripe



Any Techie
@any_techie



Any Techie

@any_techie

Why Use **Stripe**?

- Supports one-time payments & subscriptions
- Secure & developer-friendly API
- Trusted by thousands of companies
- Multi-currency support: **Stripe** allows businesses to process payments in over 110 different countries without the need for separate merchant facilities.
- No setup or monthly fees: **Stripe** offers a cost-effective solution with no setup costs, monthly fees, or minimum charges.
- International payments: **Stripe** simplifies international transactions, allowing businesses to cater to a global customer base effectively.
- And Many More....



Any Techie

@any_techie

Challenges in **Stripe** Integration

- **Secure handling of transactions**
- **Setting up webhooks for real-time updates**
- **Managing payment failures smoothly**
- **Complexity in testing and debugging: Stripe's API can be complex to work with, making it difficult to test and debug issues.**
- **Customization and flexibility: Some businesses may find Stripe's pre-built features and templates limiting, requiring custom development to meet their specific needs.**



Any Techie
@any_techie

Step-by-Step Stripe Setup

• 1

```
1 //Install Stripe SDK
2 npm install @stripe/stripe-react-native
3 cd ios && pod install # For iOS
4
```

• 2

```
1 //Wrap App in StripeProvider
2 import { StripeProvider } from '@stripe/stripe-react-native';
3 export default function App() {
4   return (
5     <StripeProvider publishableKey="your-publishable-key">
6       <YourAppNavigator />
7     </StripeProvider>
8   );
9 }
10
```

3





Any Techie

@any_techie

3

```
1 //Backend: Create Payment Intent
2 const stripe = require('stripe')('your-secret-key');
3 app.post('/create-payment-intent', async (req, res) => {
4   const { amount, currency } = req.body;
5   const paymentIntent = await stripe.paymentIntents.create({ amount, currency });
6   res.send({ clientSecret: paymentIntent.client_secret });
7 });
8
```

4

```
1 //Handling Payments on the Frontend
2 import { useStripe } from '@stripe/stripe-react-native';
3 const CheckoutScreen = () => {
4   const { initPaymentSheet, presentPaymentSheet } = useStripe();
5   const initializePayment = async () => {
6     const response = await fetch('YOUR_BACKEND_URL/create-payment-intent', { method: 'POST' });
7     const { clientSecret } = await response.json();
8     const { error } = await initPaymentSheet({ paymentIntentClientSecret: clientSecret });
9     if (!error) {
10       await presentPaymentSheet();
11     }
12   };
13   return <Button title="Pay Now" onPress={initializePayment} />;
14 };
15
```



Any Techie
@any_techie

5

```
1 //Setting Up Stripe Webhooks
2 app.post('/webhook', express.raw({ type: 'application/json' })), (req, res) => {
3   const event = req.body;
4   if (event.type === 'payment_intent.succeeded') {
5     console.log('Payment successful!');
6   }
7   res.json({ received: true });
8 });
9
```

5





Any Techie

@any_techie

Best Practices & Key Takeaway

-  **Test Webhooks before deployment**
-  **Use Stripe test cards for transactions**
-  **Never expose API keys in frontend code**
-  **Optimize API calls to reduce latency**



Any Techie
@any_techie

What's Your Experience?

Have you integrated **Stripe** in **React Native**?
What **challenges** did you face?



Drop a comment & share your thoughts!

7



THANK YOU



Follow for more



Any Techie

 @any_techie

 @anytechie